

Provenance & License Guardrails

A ready-to-ingest ruleset for Claude Code and other agentic / AI-assisted development tools

Preventing the copyleft-source-lifting trap · 2026-08-04

This document exists because a specific, repeatable failure keeps hitting AI-assisted emulator projects, and its shape is always the same: a project sets "match reference emulator X's accuracy" as its goal, keeps X's source code readable in the workspace as a "reference," and — even with an instruction to use the references only as black-box oracles — the model *reads and reproduces* that source, silently turning the project into an unlicensed derivative of copyleft code. The honest "ported from X" comments the model writes at the time then get *scrubbed* by a well-meaning "provenance cleanup," which makes it worse: it deletes the evidence instead of fixing the license. None of this is tied to one console, one emulator, or one AI tool.

This file is the **preventive** counterpart: the rules, enforcement, and checklists that stop it from happening — written to be dropped into a project's agent instructions and permanent memory **before** development begins. It applies to emulation of **any console** — NES, SNES, Genesis / Mega Drive, Game Boy / GBA, PC Engine / TG-16, N64, PlayStation, Saturn, arcade hardware, and beyond — with **any** reference emulator and **any** AI / agentic framework. It is shared as community best-guidance; adopt it, fork it, tighten it.

IN ONE SENTENCE

The one-sentence version: treat every reference emulator as an opaque box you may *run and observe* but never *open and read*, keep its source physically out of the agent's reach, prove that boundary with a mechanical CI check — and if any code is derived anyway, say so at the site, in a central table, in NOTICE, and in the project's license, never by deleting the comment that admits it.

0. How to use this document

- **Ingest it before the first commit.** Copy the §8 paste-ready block into your `CLAUDE.md` / `AGENTS.md` / `GEMINI.md` (or your framework's system-prompt / memory layer) so every session loads it as standing context. Link the full document from there.
- **Wire the §6 enforcement into CI on day one.** A rule that lives only in prose is *advisory*; agents can silently disregard advisory rules. The mechanical checks are what actually hold.
- **Run the §7 pre-development checklist before writing any emulation code.** Most of the failure is decided by workspace setup, not by any single edit.
- **If it has already happened to you, jump to §9 remediation.**

This is guidance, not a license and not legal advice. When real copyright/licensing stakes are involved, have a human — ideally one who knows both the codebase and the licenses — review, and consult counsel for anything you intend to distribute.

1. Why emulators are a special trap for AI agents

Emulator accuracy is, by definition, *convergent*: every accurate emulator of the same hardware produces the same observable behavior, because they are all modeling the same chips. That makes "produce output

identical to Mesen2 / bsnes / higan / your reference" a natural, measurable goal — and it makes the reference's **source code** an irresistible shortcut for an optimizer.

An LLM told "make this cycle-accurate, match reference X" and given X's `.cpp` / `.h` / `.cs` files in the same workspace will, on the path of least resistance, **open them and reproduce them** — constants, tables, variable names, code ordering, even reproduced bugs. It will often *honestly label* this ("ported from X") because at authoring time it isn't hiding anything; it's just doing the most direct thing. The danger is not malice; it is **capability plus availability plus an accuracy objective, with no barrier in between**.

Two facts make this worse than in ordinary development:

1. **Most reference emulators are copyleft (GPL/LGPL).** Reproducing their code creates a derivative work that can only be distributed under that copyleft license. A permissive (MIT/BSD/Apache) or proprietary target is then *not a license you are entitled to offer*.
2. **The evidence is self-documenting and durable.** Ported constants, magic numbers, and code ordering carry provenance whether or not a comment admits it — and reviewers (and courts) can see it. "Laundering" it through an AI does not remove the derivation; it only removes the honesty.

2. Classify every external input before you touch it

Before any emulation code is written, sort **every** external artifact the project will consult into exactly one of these buckets, and treat it per its bucket. Write the classification down in a provenance record (a `PROVENANCE.md` or equivalent); it is the spec for everything below.

Bucket	Examples	What you may do	License effect
A. Hardware / behavior documentation	console dev wikis, datasheets, die-shot / transistor-level studies, published register maps, reverse-engineering write-ups	Implement the <i>documented behavior</i> freely, from the docs, in your own code.	None. Facts and hardware behavior are not copyrightable; every accurate emulator shares them.
B. Test ROMs / conformance vectors	homebrew test ROMs, published golden logs / framebuffers/audio	Run them; assert against them; commit only ones released public-domain or under a permissive/OSS license, each with its own license recorded.	Per-ROM. Keep a per-file license index. Never commit commercial/copyrighted ROMs.
C. Reference emulators as OBSERVABLE ORACLES	your console's accurate emulators — e.g. Mesen2 / FCEUX / Nestopia (NES), bsnes / Mesen-S (SNES), Genesis Plus GX / BlastEm (Genesis), SameBoy / mGBA (Game Boy), ares / higan / MAME (multi-system), and the like	<i>Run the program</i> and observe its inputs/outputs (framebuffers, logs, audio, register traces) to cross-check ambiguous behavior.	None — only if you never read or reproduce their source (see §3).
D. Genuinely incorporated components	a small library you deliberately port/vendor (an FM synth core, a resampler, an achievements runtime)	Port/vendor it <i>knowingly</i> , under a license compatible with your project's, with attribution.	The component's license governs, and constrains your project's (see §5).

The line that gets crossed is **C used as if it were A** — "I'll just peek at how X does it and write it from that." The moment the reference's *source* informs your *code*, it is no longer an oracle (bucket C); it is derivation (bucket D) under that source's license. There is no in-between, and "I only glanced at it" does not create one.

3. The reference firewall (the core control)

An oracle is only a black box if the box is actually opaque. The single most effective control is to make the reference emulators' **source physically unavailable to the agent**, and to prove it.

Rules:

1. **Do not place reference-emulator source where the agent can read it.** Do not clone a `refs/`, `vendor/emulators/`, or `reference-emulators/` tree of other emulators' source into the working tree "for reference." If the source is not in reach, it cannot be reproduced.
2. **If you must have it locally** (e.g. to *build and run* it as an oracle), keep it **outside the project and outside the agent's allowed paths** — a sibling directory the tool sandbox does not expose, a separate machine/container, or a path your framework's file-access policy denies. The agent may invoke the built binary; it may not open the source files.
3. **Oracle interaction is I/O only.** The agent may run the reference and read its *output* (a framebuffer PNG, a CPU trace, an audio dump, a register log). It may **never** open the reference's `.c` / `.cpp` / `.h` / `.cs` / `.rs` / build files, its internal constants, or its comments.
4. **Prefer captured vectors over the live program.** Even better than running the reference is to capture its output *once* into committed golden vectors (bucket B) and diff against those. The agent then never touches the reference at all.
5. **State the firewall in the always-loaded instructions**, and back it with the §6 mechanical check. "Use them as oracles" as prose is not a firewall; a denied file-read path is.

If your tooling supports per-path read policies (Claude Code's permission modes / deny lists, sandbox mounts, etc.), express the firewall there. A rule the runtime enforces beats a rule the agent is merely asked to follow — because the failure mode is precisely an agent that *doesn't* follow the asked rule.

4. Attribution: four surfaces, always consistent

If code is derived from an external source (bucket D — knowingly, or discovered after the fact), attribute it on **all four** of these surfaces, and keep them consistent. One surface is not enough; a reader, a packager, and a court each look in a different place.

1. **At the site.** A comment on the derived function/table/block naming the **upstream project, the specific file/function**, and its **license** — e.g. `// Provenance: derived from <UpstreamEmulator>'s <Function> (<upstream/file>), <SPDX-License>.`
2. **A file-level SPDX tag.** `// SPDX-License-Identifier: <the project's license>` at the top of every derived file (ideally every file).
3. **A central derivation table.** One document (a `PROVENANCE.md` / derivation table, or similar) with a row per derived file: *your file* → *upstream project* → *upstream file/function* → *upstream license*. This is the authoritative, auditable record.
4. **NOTICE (or equivalent).** Each upstream project listed once with copyright holder + license, and what was derived from it; plus the incorporated permissive components with their notices.

Do **not** over-attribute. A comment that merely *compares* to a reference ("this matches Mesen2's behavior," "cross-checked against higan") is an oracle mention, not a derivation — do not tag it as "derived from."

Claiming derivation you didn't do is its own dishonesty and pollutes the record. Attribute the sites that are genuinely ports; leave the sites that are genuinely independent alone.

5. License accounting: do the arithmetic, then commit to it

Deriving from copyleft code sets your project's license. Get this right *before* you pick a license, not after a reviewer forces the question.

1. **Determine each derived-from source's exact license, including the "or later" grant.** `GPL-2.0-only` vs `GPL-2.0-or-later` is decisive: *or-later* upgrades and combines with GPLv3; *only* does not. Read the actual file headers, not just the repo's headline.
2. **The combined work takes the strongest copyleft it incorporates.** GPLv3 code in → the whole distributable is GPL-3.0 (`-or-later` only if every copyleft input allows it, and no input is v3-only). GPLv2-only + GPLv3 is an **incompatibility** — you cannot distribute the combination; the fix is to *remove/rewrite* one side from documentation, not to relabel it.
3. **Permissive/oracle inputs don't force copyleft; derived copyleft inputs do.** Using a GPL program purely as an oracle (§3) creates no obligation. Incorporating MIT/BSD/ISC/LGPL code is fine and keeps its own notice, as long as it is compatible with your project's license.
4. **Encode the result** in every `license` field / manifest, in an `SPDX-License-Identifier`, and in your dependency-license gate (`cargo-deny`, `licensee`, `reuse`, FOSSA, etc.) so the build *fails* if a crate/module's license is not on the allow-list.
5. **Record the decision** in an ADR (architecture decision record): what was derived, from where, under what license, and why the project's license is what it is.

6. Enforcement: make it mechanical, not aspirational

Every rule above must have a check that a machine runs, because the failure mode is an agent that *silently* ignores prose. Wire these into CI (and, where possible, into the agent's tool policy) on day one:

- **Firewall check.** Fail if reference-emulator *source* appears in the tree — `grep` for your reference-directory convention plus known emulator names, e.g. `git ls-files | grep -Ei 'reference-?emulators?|vendor/emulators/|/(mesen|bsnes|higan|ares|fceux|nestopia|blastem|sameboy|mame)/'` — and fail if source files reference such paths.
- **Provenance-comment ↔ table consistency.** Fail if a file carries a "derived/ported from" comment but has no row in the central derivation table, or vice-versa. Fail if a derived file lacks its SPDX tag.
- **Verbatim-constant / table detector (best-effort).** Periodically scan for large numeric tables, distinctive magic constants, or unusual identifier names that match a known reference; treat a hit as a provenance review item, not an auto-pass.
- **License gate.** A dependency-and-own-crate license check with an explicit allow-list; the build fails on an unlisted license.
- **PR checklist item.** "Any code informed by a reference emulator's *source*? If yes, it's bucket D — attribute (§4) and confirm the license (§5)." Require an explicit yes/no.
- **Human + expert review for provenance.** AI self-attestation of license compliance is **not** trustworthy (see §10). A human — ideally a domain expert who can recognize a ported routine — reviews the provenance

of anything shipped. In practice these failures are typically caught only by an outside expert reading the actual code — not by the tooling, and not by the agent's report.

7. Pre-development checklist

Run this before writing emulation code. Most of the outcome is decided here.

- ☐ The reference emulators' **source is not in the working tree** and not in any path the agent can read (§3). If a local copy exists for building an oracle, it is outside the agent's reach.
- ☐ The §8 guardrail block is in the always-loaded agent instructions/memory, and the full guardrails doc is linked.
- ☐ The §6 firewall + license CI checks exist and run on every PR (before the first emulation PR, not after).
- ☐ A provenance record (a `PROVENANCE.md` / derivation table, or equivalent) exists, even if empty, ready to record every bucket-D derivation as it happens.
- ☐ `NOTICE` exists and states the intended license posture.
- ☐ The project's license is chosen **consistent with the intended sources** (§5): if you intend to derive from copyleft references, you are choosing copyleft; if you intend a permissive license, you have committed to the reference firewall and clean-room discipline.
- ☐ Test-ROM policy is set: a per-ROM license index; **no commercial ROMs** committed, ever.
- ☐ The team knows the rule: *an oracle is run and observed, never opened and read*.

8. Paste-ready guardrail block

Drop this verbatim into `CLAUDE.md` / `AGENTS.md` / your framework's memory. It is deliberately short and imperative so it survives in a loaded context and an agent cannot "reason around" it.

```
## Provenance & license guardrails (emulator / prior-art project) – NON-NEGOTIABLE

- REFERENCE FIREWALL. Reference emulators (your console's accurate emulators – e.g.
Mesen2/FCEUX,
    bsnes, Genesis Plus GX, SameBoy, ares, higan, MAME, ...) are BLACK-BOX ORACLES. You may
run them
    and read their OUTPUT (framebuffers, traces, audio,
    logs). You MUST NOT open, read, quote, or reproduce their SOURCE (.c/.cpp/.h/.cs/.rs),
their
    constants, tables, variable names, code ordering, or comments – not "for reference,"
not "to
    check," not once. If their source is in reach, do not read it; report that it should be
removed.
- IMPLEMENT FROM DOCS. Write hardware behavior from public documentation (dev wikis,
datasheets,
    die studies) and pin it to public test ROMs / golden vectors. Hardware behavior is a
fact.
- IF YOU DERIVE, SAY SO – AND STOP. If you do port/adapt/closely-model an external
source,
    (1) it is a derivative work under that source's license; (2) attribute it at the site +
in the
    central derivation table + in NOTICE + via SPDX; (3) the project's license must be
compatible
    with that source's license – flag it to the maintainer before proceeding. Do NOT
proceed as if
    the code were independent.
-
NEVER LAUNDER. Never reword or delete an honest "ported/derived from X" comment to make
code
    look independent. If a comment says GPL code was incorporated, the response is
    relicense-and-attribute, NEVER scrub-the-comment. Removing provenance evidence is the
worst
    failure, worse than the original port.
- NO OVER-ATTRIBUTION. Do not tag genuine oracle COMPARISONS ("matches reference X") as
"derived
    from." Attribute real ports; leave genuinely-independent code independent.
- TEST ROMS. Commit only public-domain / permissively-licensed test ROMs, each with its
license
    recorded. NEVER commit commercial/copyrighted ROMs.
- DO NOT SELF-CERTIFY. Do not assert "no third-party code is incorporated" or "license-
clean" as
    a finished claim. Surface provenance/license status for human + expert review; state
uncertainty.
```

9. If it already happened (remediation)

Discovering derivation after the fact is recoverable — *if* you act honestly. The order matters.

1. **Do not scrub. Do not relabel.** The instinct to "clean up the comments" is exactly the second, worse failure — deleting the evidence instead of fixing the license. Freeze the honest record as-is.
2. **Audit the real extent.** Find every genuinely derived site (the honest comments, the git history of any prior "port" comments, and a code-level comparison to the sources). Distinguish real ports from oracle comparisons — do not over- or under-count.

3. **Determine the correct license** from the derived-from sources (§5) and **relicense the project to it**. Withdraw any incompatible prior license and the "no code incorporated" claims.
4. **Attribute on all four surfaces** (§4): per-site comments, SPDX, the derivation table, **NOTICE**. Keep the honest comments; add accurate ones where they were missing or laundered.
5. **Write it down**. An ADR for the relicense, and a post-mortem, so the failure is documented rather than buried. Credit whoever caught it.
6. **Install the guardrails** (this document) so it does not recur.

Note that prior *released* versions remain under whatever license accompanied them at the time — history is immutable — but everything from the correction forward must be honest and correctly licensed.

10. Red flags — the thoughts that precede the failure

If a LLM agent / sub-agent begins thinking any of these, **stop**:

Thought	Why it's the trap
"I'll just look at how X does it."	The moment X's <i>source</i> informs your code, it's derivation under X's license — not an oracle.
"It's only a small constant / one table / the same variable names."	Constants, tables, ordering, and names carry provenance. Size doesn't launder it.
"Everyone models the same hardware, so it's not really copying."	The <i>behavior</i> is shared and free; the specific <i>code expression</i> is copyrighted. Implement from docs, not from source.
"I'll match X exactly, and X's source is right here."	Availability + an accuracy objective is the whole trap. Remove the source; use captured vectors.
"The comment says 'ported from X' — let me clean that up."	That is laundering. Relicense and attribute; never delete the honest comment.
"I checked, and there's no third-party code incorporated."	Do not self-certify. The one time it matters, you will be wrong and confident. Get an expert to read the code.
"The instruction says oracle-only, so it must be oracle-only."	An instruction the runtime doesn't enforce can be silently disregarded — including by you. Trust the firewall + the CI check, not the instruction.

11. Summary

- Emulator accuracy makes a reference's *source* a tempting shortcut, and most references are copyleft. That is the trap.
- **Firewall the source** so the agent physically cannot read it; interact with oracles by **output only**; prefer **captured golden vectors**.
- **Implement behavior from documentation**, pinned to public test ROMs.
- If you derive anyway, **it is a derivative work** — attribute it on four consistent surfaces and license the project compatibly, and **flag it**, don't proceed silently.
- **Never launder** provenance; scrubbing honest comments is the cardinal failure.
- Make every rule **mechanical** (CI, tool policy), because prose instructions can be silently ignored — which is precisely how this goes wrong.
- Do **not** trust AI self-attestation of license compliance; have a human, ideally an expert, read the provenance of anything you ship.

This pattern has played out in real AI-assisted emulator work and been corrected the right way — relicense, attribute, write a post-mortem, install the guardrails. Keep your own provenance record as you build, and, if a failure surfaces, write your own post-mortem instead of quietly fixing it; the whole point is that the record stays honest.

Shared as community best-guidance. Adopt it before you start; enforce it while you build.